

Assignment 1:

Exploring OpenGL Programming

NAME: YIXUAN YAN
STUDENT NUMBER: 2023533152
EMAIL: YANYX2023@SHANGHAITECH.EDU.CN

1 INTRODUCTION

This assignment focuses on implementing core components of a ray-tracing renderer, including ray-triangle intersection tests, BVH (Bounding Volume Hierarchy) acceleration structures, direct illumination integrators, refractive materials, and anti-aliasing. The final goal is to render a Cornell Box scene with physically plausible lighting and refraction effects.

Completed tasks include:

- Implement ray-triangle and ray-AABB intersection tests;
- Construct a BVH acceleration structure to optimize ray-tracing efficiency;
- Develop a direct illumination integrator supporting refractive materials;
- Integrate shadow testing and multi-sample anti-aliasing;
- Validate the implementation by rendering the Cornell Box scene.

2 IMPLEMENTATION DETAILS

2.1 Task 1: Geometric Intersection Tests

Implement ray-triangle and ray-AABB (Axis-Aligned Bounding Box) intersection tests, with relevant code located in `src/accel.cpp`.

2.1.1 Ray-Triangle Intersection (`TriangleIntersect`). Implemented using the geometric method:

```
bool TriangleIntersect(const Ray& ray,
    const Vec3f& v0, const Vec3f& v1,
    const Vec3f& v2,
    Float& t, Float& u, Float& v) {
    Vec3f edge1 = v1 - v0;
    Vec3f edge2 = v2 - v0;
    Vec3f h = Cross(ray.d, edge2);
    Float a = Dot(edge1, h);
    if (a > -1e-8 && a < 1e-8) return false;
    // Ray is parallel to the triangle plane
    Float f = 1.0f / a;
    Vec3f s = ray.o - v0;
    u = f * Dot(s, h);
    if (u < 0.0 || u > 1.0) return false;
    Vec3f q = Cross(s, edge1);
    v = f * Dot(ray.d, q);
    if (v < 0.0 || u + v > 1.0) return false;
    t = f * Dot(edge2, q);
    return t > 1e-8; // Exclude reverse rays
}
```

```
}
```

Core logic: Calculate the intersection point of the ray with the triangle plane using cross products, then verify if the point lies within the triangle using barycentric coordinates.

2.1.2 Ray-AABB Intersection (`AABB::intersect`). Implemented using the Slab method:

```
bool AABB::intersect(const Ray& ray,
    Float& t_min, Float& t_max) const {
    for (int i = 0; i < 3; ++i) {
        Float invD = 1.0f / ray.d[i];
        Float t0 = (min[i] - ray.o[i]) * invD;
        Float t1 = (max[i] - ray.o[i]) * invD;
        if (invD < 0.0f) std::swap(t0, t1);
        t_min = std::max(t0, t_min);
        t_max = std::min(t1, t_max);
        if (t_min > t_max) return false;
    }
    return t_max > 1e-8;
}
```

Core logic: Compute intersection points of the ray with the AABB along the three axes, then determine if a valid intersection interval exists by comparing the maximum near intersection and minimum far intersection.

2.2 Task 2: BVH Construction and Traversal

Implement BVH tree construction (`BVHTree<_>::build` in `bvh_tree.h`) using a recursive partitioning strategy:

```
template <typename Primitive>
void BVHTree<Primitive>::build(
    std::vector<Primitive>& primitives) {
    root = std::make_unique<BVHNode>();
    root->primitives = std::move(primitives);
    // Compute node bounding box
    root->bounds = ComputeBounds(root->primitives);
    if (root->primitives.size() <= max_leaf_size)
        return;

    // Select partitioning axis
    int axis = root->bounds.maxExtent();
    // Sort primitives along the axis
    std::sort(root->primitives.begin(),
        root->primitives.end(),
        [axis](const Primitive& a, const Primitive& b) {
            return a.getBounds().centroid()[axis] <
                b.getBounds().centroid()[axis];
        });
    // Partition primitives
    auto mid = root->primitives.begin() +
```

1:2 • Name: Yixuan Yan

student number:2023533152

email: yanyx2023@shanghaitech.edu.cn

```
root->primitives.size() / 2;
std::vector<Primitive> left_primitives(
root->primitives.begin(), mid);
std::vector<Primitive> right_primitives(
mid, root->primitives.end());
root->primitives.clear(); // Internal nodes do not
    store primitives

// Recursively build subtrees
root->left = std::make_unique<BVHNode>();
buildRecursive(root->left, std::move(left_primitives));
root->right = std::make_unique<BVHNode>();
buildRecursive(root->right, std::move(right_primitives)
);
}
```

Core logic: 1. Compute the bounding box for the node; 2. If the number of primitives exceeds the threshold, partition along the longest axis of the bounding box; 3. Recursively build left and right subtrees to form the BVH hierarchy.

2.3 Task 3: Direct Illumination Integrator with Shadow Testing

Implement the `IntersectionTestIntegrator` (in `src/integrator.cpp`) to handle ray tracing, direct illumination calculation, and shadow testing.

2.3.1 Multi-Sample Anti-Aliasing (`IntersectionTestIntegrator::render`).

Generate multiple samples per pixel to reduce aliasing:

```
for (int dx = 0; dx < resolution.x; dx++) {
    for (int dy = 0; dy < resolution.y; dy++) {
        for (int sample = 0; sample < spp; sample++) {
            Vec2f pixel_sample = sampler.getPixelSample();
            // Sub-pixel offset
            auto ray = camera->generateDifferentialRay(
                pixel_sample.x, pixel_sample.y);
            Vec3f L = Li(scene, ray, sampler);
            camera->getFilm()->commitSample(
                pixel_sample, L);
        }
    }
}
```

2.3.2 Ray Tracing and Material Handling (`IntersectionTestIntegrator::Li`). Trace rays and handle refractive materials:

```
Vec3f IntersectionTestIntegrator::Li(
    ref<Scene> scene, DifferentialRay& ray,
    Sampler& sampler) const {
    Vec3f color(0.0);
    DifferentialRay current_ray = ray;
    for (int i = 0; i < max_depth; ++i) {
        SurfaceInteraction isect;
        if (!scene->intersect(current_ray, isect)) break;

        isect.wo = -current_ray.direction;
        auto* bsdf = isect.bsdf;
        if (auto* refract = dynamic_cast<const
            PerfectRefraction*>(bsdf)) {
            // Handle refraction
```

```
Float pdf;
Vec3f wi = refract->sample(isect, sampler, &pdf);
Vec3f spawn_origin = isect.p + wi * 1e-3f;
// Avoid self-intersection
current_ray = DifferentialRay(spawn_origin, wi);
continue;
}

if (dynamic_cast<const IdealDiffusion*>(bsdf)) {
    // Compute direct illumination
    color = directLighting(scene, isect);
    break;
}
break;
}
return color;
}
```

2.3.3 Direct Illumination and Shadow Testing (`IntersectionTestIntegrator::directLighting`). Calculate diffuse illumination and check for shadows:

```
Vec3f IntersectionTestIntegrator::directLighting(
    ref<Scene> scene, SurfaceInteraction& isect) const {
    Vec3f light_dir = Normalize(
        point_light_position - isect.p);
    Float dist_to_light = Norm(
        point_light_position - isect.p);
    // Generate shadow ray
    Vec3f shadow_origin = isect.p + isect.normal * 1e-3f;
    DifferentialRay shadow_ray(
        shadow_origin, light_dir, 1e-3f,
        dist_to_light - 1e-3f);
    SurfaceInteraction shadow_isect;
    if (scene->intersect(shadow_ray, shadow_isect))
        return Vec3f(0.0); // Occluded

    // Lambertian diffuse calculation
    auto* diffuse_bsdf = dynamic_cast<const IdealDiffusion
        *>(isect.bsdf);
    Vec3f albedo = diffuse_bsdf->evaluate(isect);
    Float cos_theta = std::max(
        Dot(light_dir, isect.normal), 0.0f);
    Float attenuation = 1.0f / (dist_to_light *
        dist_to_light);
    // Inverse-square law
    return albedo * cos_theta * point_light_flux *
        attenuation * 0.1f;
}
```

2.4 Task 4: Perfect Refraction Material

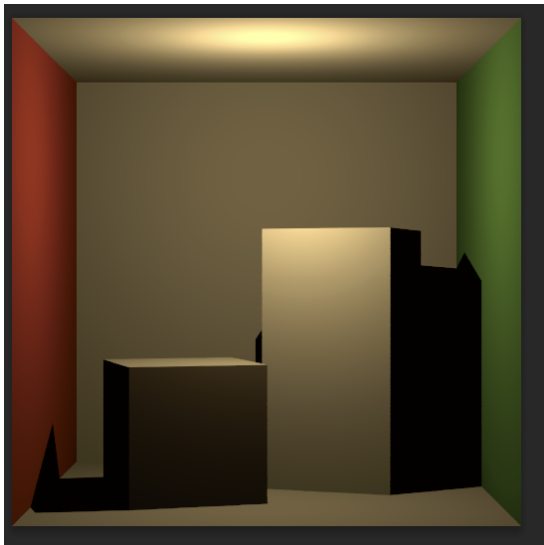
Implement Snell's Law to compute refraction directions (in `src/bsdf.cpp`):

```
Vec3f PerfectRefraction::sample(
    SurfaceInteraction& isect, Sampler& sampler,
    Float* pdf) const {
    Vec3f n = isect.shading.n;
    Float cosi = Dot(isect.wo, n);
    bool entering = cosi > 0;
    Float eta_i = entering ? 1.0f : eta;
    Float eta_t = entering ? eta : 1.0f;
```

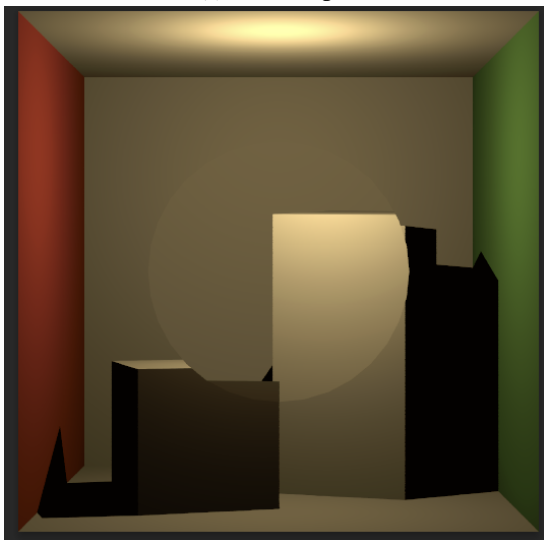
```
if (!entering) n = -n;

Vec3f refracted_dir;
if (!Refract(isect.wo, n, eta_i / eta_t, refracted_dir))
{
    refracted_dir = Reflect(isect.wo, n);
    // Total internal reflection
}
isect.wi = refracted_dir;
if (pdf) *pdf = 1.0f;
return Vec3f(1.0f);
}
```

3 RESULTS



(a) cbox_no_light



(b) cbox_no_light_refract